

ANLP Assignment-1 Report

Name: Mohd Zeeshan

Roll: 2025901020

 huggingface.co/Zappy-AI/anlp-a1

 W&B wandb.ai/mohd-zeeshan-iiit-hyderabad/anlp-assignment1

 https://github.com/zappy586/ANLP_Assignment_1

1. Introduction

Task: This assignment involves a cipher-text to plain-text task using transformer [1] model variations and their results on it.

This report contains the ablations that test and compare architectural changes to the transformer from recent literature, using the standard transformer as a baseline. The different architectural ablations performed in this report are:

1. Standard Multi-Headed Transformer encoder-decoder - C1
2. Transformer with Rotary Positional Embeddings (RoPE) - C2
3. Transformer with Grouped Query Attention (GQA) - C3
4. Transformer with RMSNorm - C4
5. Byte Latent Transformer (Tokenizer-free) - C5

2. Architecture Overview

We use the exact same encoder-decoder architecture configurations in C1-C4 to ensure consistency across experiments. The values are derived from the original Transformer paper [1].

Configuration	Model Dimension	Number of Layers	Number of Heads
C1	512	6	8
C2	512	6	8
C3	512	6	8
C4	512	6	8

Byte Latent Transformer (BLT) is slightly different from other ablations as it involves separate encoder-decoder modules that directly work on bytes and convert them to latent patches for the main transformer block. Hence we have 2 extra parameters:

Configuration	Model Dimension (Main Transformer)	Number of Layers (Main Transformer)	Number of Heads (Main Transformer)	Local Encoder Layers	Local Decoder Layers
C5	512	6	8	2	2

3. Experimental Setup

Tokenization and sequence lengths: C1-C4 configurations use a BPE tokenization scheme, with a vocabulary of 256 tokens for input and 512 tokens for output. We train and run the input tokenizer directly over the cipher bits (not bytes). Both tokenizers are extended by 3 more tokens: <pad>, <sos> and <eos>.

As for C5, we don’t use a tokenizer, rather we have separate encoder and decoder modules that convert the individual bytes (not bits) to patches for the global transformer, and vice-versa. We use a fixed-length patching of size 1 for simplicity but the original paper used entropy-based patching using a byte-level SLM.

The max sequence length used was 320 tokens for both input and output for all 5 experiments.

Dataset: We create train-test-val splits from the dataset using a 70:15:15 ratio. Each line is considered a separate sample.

Training configuration: All 5 experiments use the same exact training configuration to maintain consistency:

1. Epochs: 20
2. Learning Rate: 3e-4
3. Batch Size: 32
4. Optimizer: AdamW
5. Loss: Cross Entropy
6. GPU: 1 x RTX 5060 Ti (16 GB)

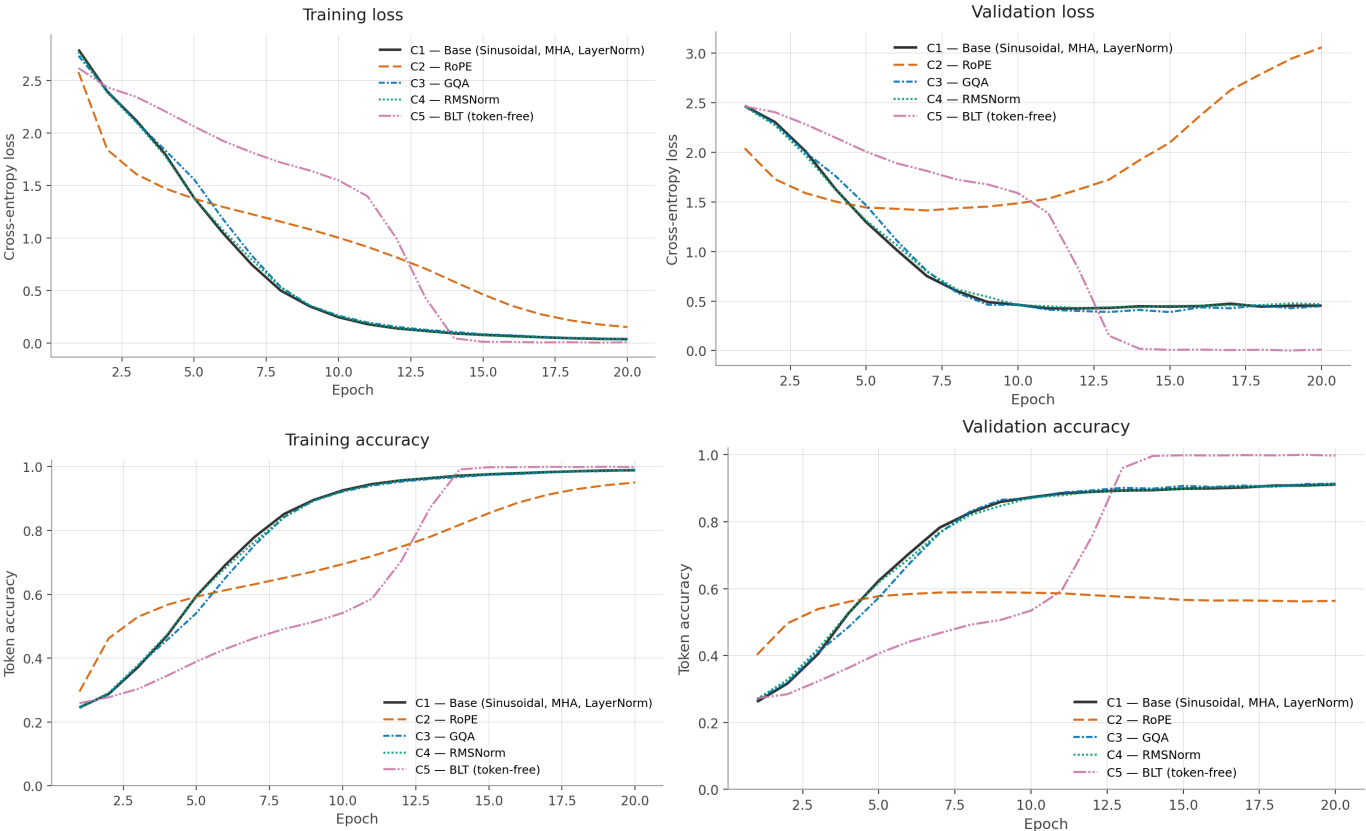
4. Results

The following tables shows the final results after training (on test set):

	Bit-level accuracy	Sequence accuracy	Levenshtein	BLEU	ROUGE
C1	91.8%	0%	36.0	0.47	0.61
C2	66.0%	0%	136.8	0.004	0.17
C3	91.4%	0%	34.6	0.47	0.62
C4	91.6%	0%	35.66	0.47	0.62
C5	99.9%	94.0%	0.12	-	-

	Parameters (M)	Training time (mins)	Peak VRAM (train-time)
C1	44.8M	18.4	9.2 GB
C2	44.8M	19.0	9.2 GB
C3	37.7M	16.1	9.1 GB
C4	44.7M	17.8	8.4 GB
C5	38.2M	19.2	9.2 GB

Training Plots:



5. Ablation Analysis

C2 (Rotary Positional Embeddings):

In C2 we replace Sinusoidal Embeddings with Rotary Positional Embeddings (RoPE) [2]. This change severely degraded the performance: bit accuracy fell from 91% to 66%. The validation accuracy peaked at around 0.58 and then regressed for the rest of the training epochs. Compute and training time however remained similar.

Likely Cause: Sinusoidal embeddings are added to the token embeddings, and therefore propagate through the residual stream and into the encoder's output hidden states, which carries positional information that decoder queries in its cross-attention layers can directly match against. RoPE on the other hand, according to the paper, rotates the keys and queries just within the self-attention computation in the encoder layer, leaving the residual stream untouched.

The source passing through the encoder consists of long tokenized sequences of 0's and 1's, while the decoder processes plaintext, both having different vocabularies, so the decoder's queries cannot identify which encoder positions correspond to a given output character. This isn't due to RoPE itself, but because the relative offset between the input and output sequences is undefined. RoPE is designed to work best with decoder-only models.

C3 (Grouped Query Attention):

GQA [3] slightly outperformed the base model (bit acc: 91.4% vs 91.8%, Levenshtein 34.6 vs 36.0) while reducing parameters by 15.8%(44.8M params to 37.7M params) and training time by 8.1% (16.8 vs 18.4 minutes).

Intuitively, sharing KV-projections across query heads should have reduced the performance of this model. And hence the most significant finding of this ablation is the absence of degradation in performance. This could be attributed to the nature of the task itself, since it may not require the full diversity of key-value subspaces. GQA, therefore, in this case is effectively a free speed boost at no accuracy cost.

C4 (RMS Normalization):

RMSNorm [4], likewise matched the baseline, with bit accuracy at 91.7% and Levenshtein 35.7, while reducing peak VRAM usage by a significant 8.2% (8.46 GB vs 9.22 GB).

Removing LayerNorm's mean-centering and the learnable bias eliminates one reduction operation per normalization layer, which makes it ~25K parameters across the model's 48 norm layers. For larger models, this could mean significant savings on VRAM usage during training at no accuracy cost.

C5 (Byte Latent Transformer):

Byte latent transformer [5] gave the best performance by large margin with 99.9% bit accuracy and sequence accuracy of 81.3%. Every other model gave 0% in sequence accuracy. This means that BLT was able to perfectly reconstruct a lot of the sequences, whereas the previous ablations had small mistakes spread across the

resulting sequences. This model also resulted in a 13.3% reduction in both VRAM usage (6.9 GB vs 9.2 GB) and in training time (18.4 minutes vs 13.8 minutes).

The result depends on a specific implementation choice: bytes are parsed from the raw bit-string ciphertext before patching (01000010 → the integer 66) rather than treating each bit-character as its own byte, and patch size is set to 1 byte. Since one byte corresponds to exactly one plaintext character, this gives a one-to-one correspondence between patches and decoder output positions, a guarantee which previous models do not enjoy. Overall, C5 shows that matching the tokenization granularity to the underlying task structure (i.e byte-level cipher-to-plain) matters more than any of the architectural changes tested in C2–C4 combined.

6. References

- [1] Ashish Vaswani, et al, "Attention Is All You Need," 2023.
- [2] Jianlin Su, et al, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2023.
- [3] Joshua Ainslie, et al, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," 2023.
- [4] Biao Zhang, Rico Sennrich, et al, "Root Mean Square Layer Normalization," 2019.
- [5] Artidoro Pagnoni, undefined., et al, "Byte Latent Transformer: Patches Scale Better Than Tokens," 2024.